

МИНОБРНАУКИ РОССИИ
ФГБОУ ВО «ИжГТУ имени М. Т. Калашникова»
Факультет «Информационные технологии»
Кафедра «Программное обеспечение»

Работа защищена с оценкой

«_____»

Дата _____

Подпись _____ / _____

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе по дисциплине «Базы данных»
на тему «Разработка базы данных для автоматизации олимпиады»

Выполнил

Студент гр. Б24-191-3

Т.И.Ишматов

Руководитель

к.т.н., доцент

А.В.Старыгин

Рецензия:

степень достижения поставленной цели работы _____

полнота разработки темы _____

уровень самостоятельности работы обучающегося _____

недостатки работы _____

Ижевск 2026

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ	3
ССЫЛКА НА КОД В ПУБЛИЧНОМ РЕПОЗИТОРИИ	4
ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ	6
1.1 Описание предметной области	6
1.2 Описание языков и средств разработки	7
1.3 Концептуальная модель базы данных	8
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ	9
2.1 Реализация базы данных	9
2.1.1 Таблицы	9
2.1.2 Представления	11
2.1.3 Индексы	14
2.1.4 Триггер	14
2.2 Клиентское приложение	15
2.2.1 Структура файлов	15
2.2.2 Подключение к базе данных	17
2.2.3 Работа приложения	18
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

В курсовой работе требуется:

Спроектировать и создать базу данных в PostgreSQL (либо иной СУБД по выбору)

Разработать клиентское приложение для работы с созданной базой данных (язык программирования любой по выбору)

Требования

В базе данных должны присутствовать:

1. Не менее 6 таблиц
2. Ограничения: DEFAULT, CHECK, PRIMARY KEY, UNIQUE, FOREIGN KEY
3. Индексы
4. Проекция (VIEW): по одной таблице, по нескольким таблицам, используя GROUP BY и HAVING,
5. Триггеры выполняющие каскадные изменения данных в связанных таблицах, либо поддерживающие денормализованные данные

Требования к клиентскому приложению:

Наличие общего интерфейса, позволяющего работать со всеми таблицами и отчётами

При работе с таблицами обеспечивается:

1. Перемещение по записям
2. Корректировка записей
3. Добавление новых записей
4. Удаление записей
5. Поиск записей по отдельным полям
6. Задание фильтра по отдельным полям
7. Задание сортировки по отдельным полям
8. Как минимум одна форма должна обеспечивать ввод данных в две таблицы с отношением 1:M

Наличие не менее 3 отчётов. Отчет:

1. формируется не менее чем из 2-х таблиц
2. содержит не менее 1-го уровня группировки,
3. вычисляемые поля
4. итоговые данные
5. перед формированием отчёта у пользователя запрашиваются фильтры и параметры сортировки по отдельным полям.

Вариант 9: Разработка базы данных для автоматизации олимпиады

ССЫЛКА НА КОД В ПУБЛИЧНОМ РЕПОЗИТОРИИ

Публичный репозиторий: GitHub

Ссылка: <https://github.com/timofey18269/kyrsovie>

/4 semestr/DB(postgres, ast.dotnet+razor_pages)

ВВЕДЕНИЕ

В рамках данной курсовой работы была разработана информационная система для хранения и обработки данных об Олимпийских соревнованиях. Основной целью проекта являлось создание базы данных и клиентского приложения для удобного просмотра, редактирования и анализа информации о спортсменах, странах, видах спорта, площадках проведения соревнований, расписании стартов и результатах участников.

В ходе выполнения работы были реализованы следующие задачи:

1. создание реляционной базы данных в PostgreSQL;
2. разработка таблиц, ограничений, внешних ключей и триггеров;
3. создание представлений для формирования отчетов;
4. разработка клиентского веб-приложения на ASP.NET Core Razor Pages;
5. реализация функций просмотра, поиска, фильтрации, сортировки и редактирования данных;
6. реализация экспорта отчетов в Excel.

В результате была создана полноценная информационная система, позволяющая удобно работать с данными Олимпийских соревнований через веб-интерфейс.

ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Описание предметной области

Олимпийские соревнования представляют собой сложную систему, включающую большое количество взаимосвязанных сущностей. Для хранения информации о соревнованиях требуется централизованная база данных, обеспечивающая удобное управление данными и возможность их анализа.

В предметной области были выделены следующие основные сущности:

- страны
- спортсмены
- виды спорта
- площадки проведения соревнований
- расписание стартов
- результаты соревнований

Каждый спортсмен представляет определённую страну и участвует в конкретном виде спорта. Соревнования проводятся на различных площадках в определённое время. По итогам соревнований спортсменам присваиваются места и результаты.

На основе хранящихся данных формируются аналитические отчёты:

- количество медалей по странам
- результаты спортсменов
- средний возраст спортсменов по видам спорта
- расписание стартов по площадкам

1.2 Описание языков и средств разработки

Для реализации проекта использовались следующие технологии.

PostgreSQL — объектно-реляционная система управления базами данных с открытым исходным кодом. Она поддерживает:

- внешние ключи
- представления
- триггеры
- индексы
- ограничения целостности

ASP.NET Core — современный фреймворк компании Microsoft для разработки веб-приложений. Он обеспечивает:

- маршрутизацию
- работу с HTTP-запросами
- подключение к базе данных
- внедрение зависимостей
- серверную обработку данных

Razor Pages — технология построения веб-интерфейсов в ASP.NET Core.

Каждая страница состоит из двух файлов:

.cshtml — HTML-разметка

.cshtml.cs — серверная логика страницы

Entity Framework Core — ORM-библиотека для взаимодействия с базой данных через объекты C#.

Она позволяет:

- выполнять запросы LINQ;
- автоматически отображать таблицы в классы;
- выполнять операции CRUD;
- работать с представлениями базы данных.

Npgsql — драйвер подключения PostgreSQL к .NET-приложениям.

Используется для взаимодействия ASP.NET Core и PostgreSQL.

ClosedXML — библиотека для создания Excel-файлов формата .xlsx.
В проекте применяется для экспорта отчётов.

1.3 Концептуальная модель базы данных

Концептуальная модель базы данных включает следующие сущности:

- Country;
- Sport;
- Participant;
- Venue;
- EventSchedule;
- Result.

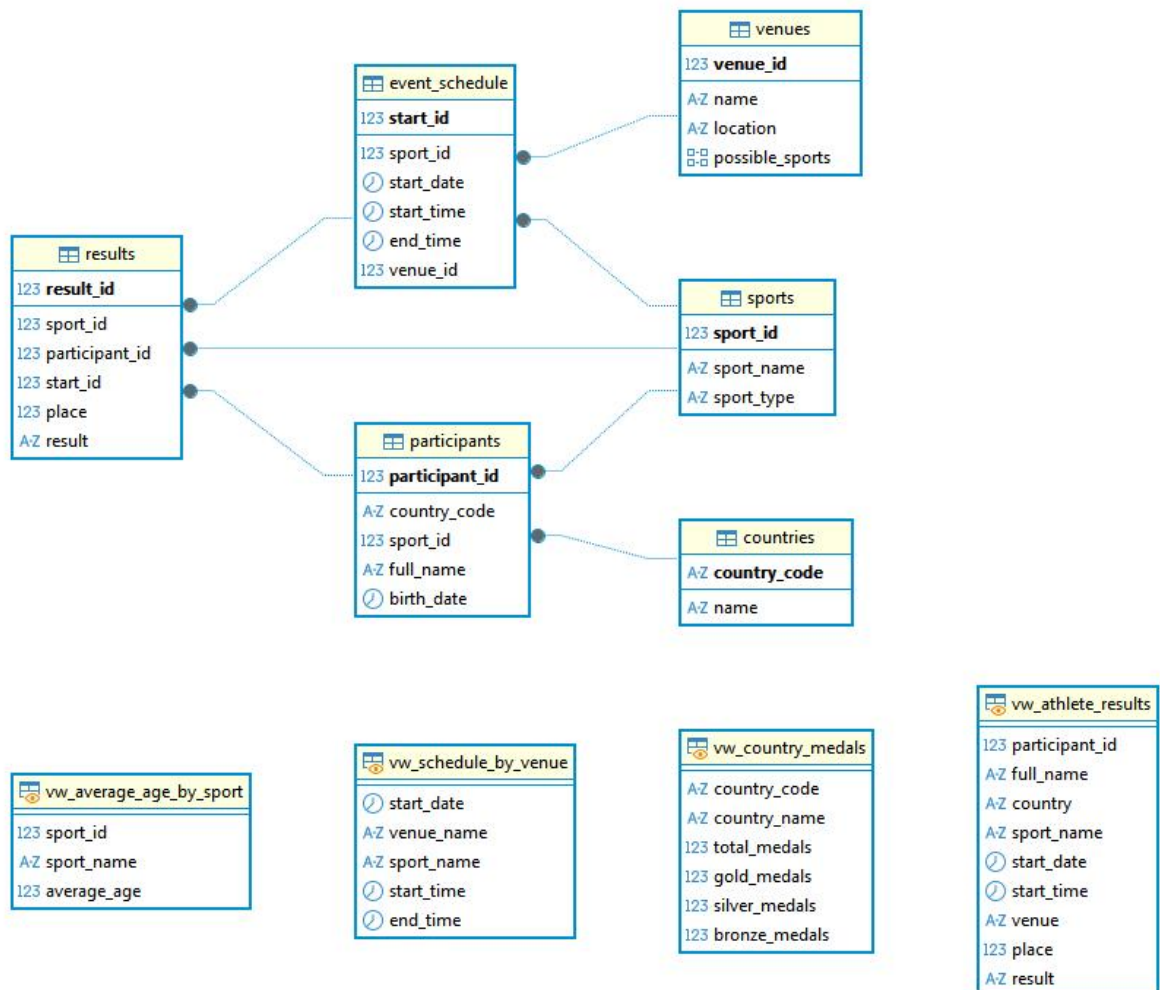
Основные связи между сущностями:

- одна страна может иметь множество спортсменов;
- один вид спорта может включать множество соревнований;
- один спортсмен может иметь множество результатов;
- одна площадка может использоваться для множества стартов;
- один старт связан с конкретным видом спорта и площадкой.

База данных реализована по реляционной модели с использованием первичных и внешних ключей.

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Реализация базы данных



2.1.1 Таблицы

Таблица Countries предназначена для хранения информации о странах. Поле CountryId является первичным ключом. Поле CountryName содержит название страны, а CountryCode хранит сокращенный международный код страны. Для поля CountryCode установлено ограничение уникальности.

```
CREATE TABLE countries (  
    country_code VARCHAR(3) PRIMARY KEY, -- ( 'RUS', 'USA'... )  
    name VARCHAR(100) NOT NULL UNIQUE  
);
```

Таблица Sports предназначена для хранения информации о видах спорта. Поле SportId является первичным ключом. Поле SportName хранит

название вида спорта, а поле Category содержит категорию спорта (индивидуальный или командный).

```
CREATE TABLE sports (  
    sport_id SERIAL PRIMARY KEY,  
    sport_name VARCHAR(100) NOT NULL,  
    sport_type VARCHAR(20) NOT NULL CHECK (sport_type IN  
    ('team', 'individual')),  
    CONSTRAINT unique_sport_type UNIQUE (sport_name, sport_type)  
);
```

Таблица Participants содержит информацию о спортсменах. Поле ParticipantId является первичным ключом. Поле CountryId представляет собой внешний ключ, связывающий спортсмена со страной.

```
CREATE TABLE participants (  
    participant_id SERIAL PRIMARY KEY,  
    country_code VARCHAR(3) NOT NULL,  
    sport_id INT NOT NULL,  
    full_name VARCHAR(200) NOT NULL,  
    birth_date DATE NOT NULL,  
    CONSTRAINT fk_participants_countries FOREIGN KEY  
    (country_code)  
    REFERENCES countries(country_code) ON DELETE RESTRICT,  
    CONSTRAINT fk_participants_sports FOREIGN KEY (sport_id)  
    REFERENCES sports(sport_id) ON DELETE RESTRICT,  
    CONSTRAINT check_age CHECK (EXTRACT(YEAR FROM  
    AGE(CURRENT_DATE, birth_date)) >= 18)  
);
```

Таблица Venues предназначена для хранения информации о спортивных объектах. VenueId является первичным ключом.

```
CREATE TABLE venues (  
    venue_id SERIAL PRIMARY KEY,  
    name VARCHAR(200) NOT NULL,  
    location VARCHAR(500) NOT NULL,  
    possible_sports INT[] DEFAULT ARRAY[]::INT[]  
);
```

Таблица EventSchedules хранит расписание соревнований. StartId - первичный ключ. Поля SportId и VenueId являются внешними ключами.

```
CREATE TABLE event_schedule (  
    start_id SERIAL PRIMARY KEY,  
    sport_id INT NOT NULL,  
    start_date DATE NOT NULL,  
    start_time TIME NOT NULL,  
    end_time TIME NOT NULL,  
    venue_id INT NOT NULL,  
    CONSTRAINT fk_event_schedule_sports FOREIGN KEY (sport_id)
```

```
REFERENCES sports(sport_id) ON DELETE RESTRICT,
CONSTRAINT fk_event_schedule_venues FOREIGN KEY (venue_id)
REFERENCES venues(venue_id) ON DELETE RESTRICT,
CONSTRAINT check_time CHECK (end_time > start_time)
);
```

Таблица Results предназначена для хранения результатов соревнований. Поле ResultId является первичным ключом. Поля SportId, ParticipantId и StartId являются внешними ключами.

```
CREATE TABLE results (
    result_id SERIAL PRIMARY KEY,
    sport_id INT NOT NULL,
    participant_id INT NOT NULL,
    start_id INT NOT NULL,
    place INT,
    result TEXT,
    CONSTRAINT fk_results_sports FOREIGN KEY (sport_id)
    REFERENCES sports(sport_id) ON DELETE RESTRICT,
    CONSTRAINT fk_results_participants FOREIGN KEY
(participant_id)
REFERENCES participants(participant_id) ON DELETE
RESTRICT,
    CONSTRAINT fk_results_event_schedule FOREIGN KEY (start_id)
    REFERENCES event_schedule(start_id) ON DELETE RESTRICT,
    CONSTRAINT unique_participant_start UNIQUE (participant_id,
start_id)
);
```

Для обеспечения корректности данных были реализованы ограничения CHECK. Например, возраст спортсменов не может быть отрицательным, вместимость спортивной площадки должна быть больше нуля, а место участника в соревновании должно быть положительным числом. Также были реализованы ограничения UNIQUE для предотвращения дублирования данных.

2.1.2 Представления

Представление CountryMedalsView используется для отображения количества золотых, серебряных и бронзовых медалей по странам. При формировании представления используются таблицы Countries, Participants и Results. В запросе используются группировка и вычисляемые поля для подсчета количества медалей.

```
CREATE OR REPLACE VIEW vw_country_medals AS
```

```

SELECT
    c.country_code,
    c.name AS country_name,
    COUNT(CASE WHEN r.place BETWEEN 1 AND 3 THEN 1 END) AS
total_medals,
    COUNT(CASE WHEN r.place = 1 THEN 1 END) AS gold_medals,
    COUNT(CASE WHEN r.place = 2 THEN 1 END) AS silver_medals,
    COUNT(CASE WHEN r.place = 3 THEN 1 END) AS bronze_medals
FROM countries c
LEFT JOIN participants p
    ON c.country_code = p.country_code
LEFT JOIN results r
    ON p.participant_id = r.participant_id
GROUP BY c.country_code, c.name
ORDER BY gold_medals DESC,
         silver_medals DESC,
         bronze_medals DESC;

```

Представление AthleteResultsView используется для отображения результатов спортсменов. Оно объединяет таблицы Participants, Sports и Results.

```
CREATE OR REPLACE VIEW vw_athlete_results AS
SELECT
    p.participant_id,
    p.full_name,
    c.name AS country,
    s.sport_name,
    es.start_date,
    es.start_time,
    v.name AS venue,
    r.place,
    r.result
FROM results r
JOIN participants p
    ON r.participant_id = p.participant_id
JOIN countries c
    ON p.country_code = c.country_code
JOIN sports s
    ON r.sport_id = s.sport_id
JOIN event_schedule es
    ON r.start_id = es.start_id
JOIN venues v
    ON es.venue_id = v.venue_id
ORDER BY p.full_name;
```

Представление AverageAgeBySportView предназначено для вычисления среднего возраста спортсменов по каждому виду спорта. Для формирования представления используются таблицы Participants, Results и Sports. В запросе используется агрегатная функция AVG.

```
CREATE OR REPLACE VIEW vw_average_age_by_sport AS
SELECT
    s.sport_id,
    s.sport_name,
    ROUND(
        AVG(
            EXTRACT(YEAR FROM AGE(CURRENT_DATE, p.birth_date))
        ),
        2
    ) AS average_age
FROM sports s
LEFT JOIN participants p
    ON s.sport_id = p.sport_id
GROUP BY s.sport_id, s.sport_name
ORDER BY average_age DESC;
```

Представление ScheduleByVenueView используется для отображения расписания соревнований по площадкам. В представлении объединяются таблицы EventSchedules, Sports и Venues.

```
CREATE OR REPLACE VIEW vw_schedule_by_venue AS
SELECT
    es.start_date,
    v.name AS venue_name,
    s.sport_name,
    es.start_time,
    es.end_time
FROM event_schedule es
JOIN sports s
    ON es.sport_id = s.sport_id
JOIN venues v
    ON es.venue_id = v.venue_id
ORDER BY es.start_date,
         v.name,
         es.start_time;
```

2.1.3 Индексы

Для повышения производительности работы базы данных были созданы индексы.

```
CREATE INDEX idx_participants_country ON
participants(country_code);
CREATE INDEX idx_participants_sport ON participants(sport_id);
CREATE INDEX idx_event_schedule_date ON
event_schedule(start_date);
CREATE INDEX idx_event_schedule_venue ON event_schedule(venue_id);
CREATE INDEX idx_results_start ON results(start_id);
CREATE INDEX idx_results_participant ON results(participant_id);
```

Создание индексов позволяет ускорить выполнение JOIN-запросов, сортировку и фильтрацию данных.

2.1.4 Триггер

В базе данных был реализован триггер для проверки того что вид спорта по которому проводится мероприятие разрешён на данной площадке.

```
CREATE OR REPLACE FUNCTION check_sport_on_venue()
RETURNS TRIGGER AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM venues
        WHERE venue_id = NEW.venue_id
        AND NEW.sport_id = ANY(possible_sports)
```

```

    ) THEN
        RAISE EXCEPTION 'Sport % is not allowed at venue %',
NEW.sport_id, NEW.venue_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_check_sport_on_venue
    BEFORE INSERT OR UPDATE ON event_schedule
    FOR EACH ROW
    EXECUTE FUNCTION check_sport_on_venue();

```

2.2 Клиентское приложение

2.2.1 Структура файлов

Файлы которые были созданы в процессе разработки (не относящиеся к стандартному набору файлов для проекта на asp.net razor_pages)

OlympiadViewer

 Data/

 OlympiadContext.cs

 Models/

 Country.cs

 Sport.cs

 Participant.cs

 Venue.cs

 EventSchedule.cs

 Result.cs

 Views/

 CountryMedalsView.cs

 AthleteResultsView.cs

 AverageAgeBySportView.cs

 ScheduleByVenueView.cs

 Services/

 DynamicQueryService.cs

 ExportService.cs

 Pages/

 Shared/

 _Layout.cshtml

 _Sidebar.cshtml

 Countries/

 Index.cshtml

Create.cshtml
Delete.cshtml
Edit.cshtml

Sports/

Participants/
 . . .

Venues/

EventSchedules/
 . . .

Results/

Reports/ . . .

Файл OlympiadContext.cs отвечает за подключение к базе данных и описание DbSet для всех таблиц и представлений.

Папка Models содержит классы моделей, соответствующие таблицам базы данных и представлениям.

Папка Services содержит сервисы для динамической фильтрации, сортировки и экспорта данных.

Во всех страницах используется общий шаблон _Layout.cshtml. Данный файл содержит общую HTML-разметку, подключение Bootstrap и отображение бокового меню.

Файл Sidebar.cshtml отвечает за отображение списка таблиц и отчетов. При выборе пункта меню пользователь переходит на соответствующую Razor Page.

Каждая таблица реализована как отдельная папка внутри Pages/Tables. Внутри каждой папки находятся страницы Index, Create, Edit и Delete.

Страница Index отвечает за отображение таблицы, сортировку, фильтрацию и поиск.

Страница Create используется для добавления новых записей.

Страница Edit используется для редактирования записей.

Страница Delete используется для удаления записей.

2.2.2 Подключение к базе данных

Подключение к базе данных реализовано через Entity Framework Core и библиотеку Npgsql. Строка подключения хранится в файле appsettings.json. В ней указывается адрес сервера PostgreSQL, имя базы данных, логин и пароль пользователя.

В файле OlympiadContext.cs описывается контекст базы данных, то есть все таблицы сопоставляются с классами(моделями), для каждой таблицы прописываются все ограничения и внешние зависимости, которые есть в базе данных.

В файле Program.cs выполняется регистрация контекста базы данных:

```
builder.Services.AddDbContext<OlympiadContext>(options =>
{
    options.UseNpgsql(
        builder.Configuration.GetConnectionString(
            "DefaultConnection"));
});
```

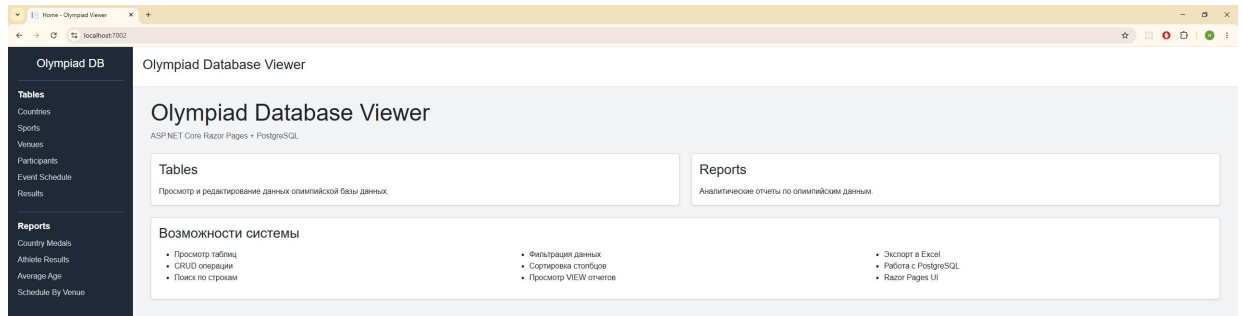
После регистрации контекста Entity Framework Core автоматически создает объект OlympiadContext и предоставляет доступ к базе данных во всех Razor Pages. Каждая страница получает контекст базы данных:

```
private readonly OlympiadContext _context;
public IndexModel(OlympiadContext context)
{
    _context = context;
}
```

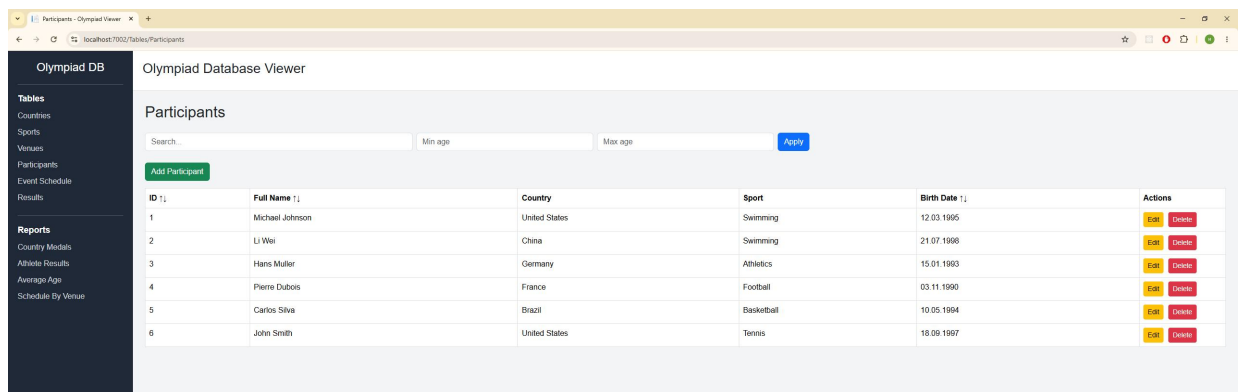
После этого к записям в базе данных можно обращаться как к созданным объектам соответствующего класса.

2.2.3 Работа приложения

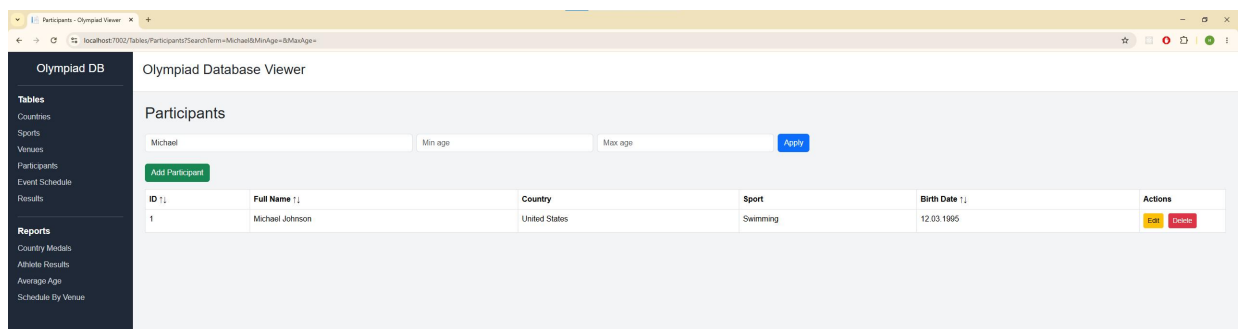
Основной страницей приложения является Index.cshtml. На ней отображается общее описание системы. Боковая панель отображается всегда.



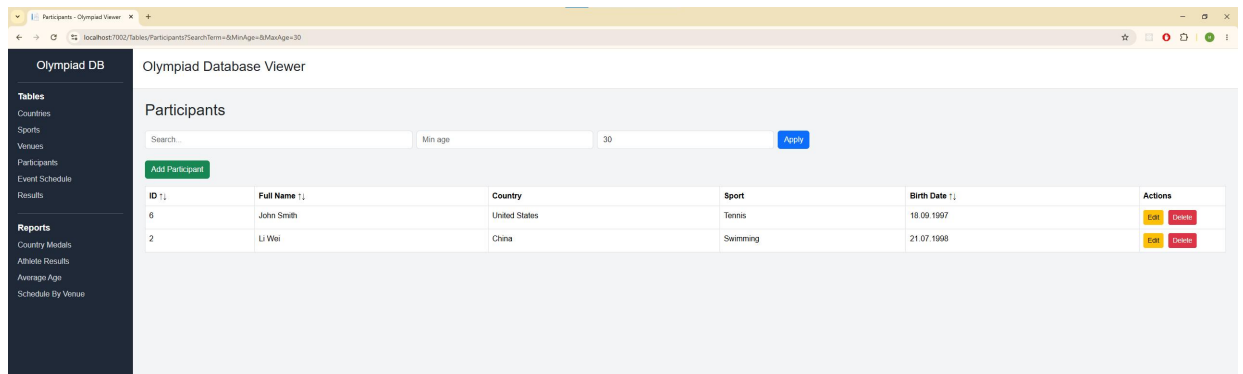
Для примера таблиц подробно рассмотрим страницу Participants. Остальные страницы для отображения таблиц обладают таким же функционалом. Данная страница отображает список всех участников соревнований, содержит информацию о спортсменах, включая имя, фамилию, пол, дату рождения и страну.



На странице реализован поиск по строковым полям. Пользователь может искать участников по имени или фамилии.



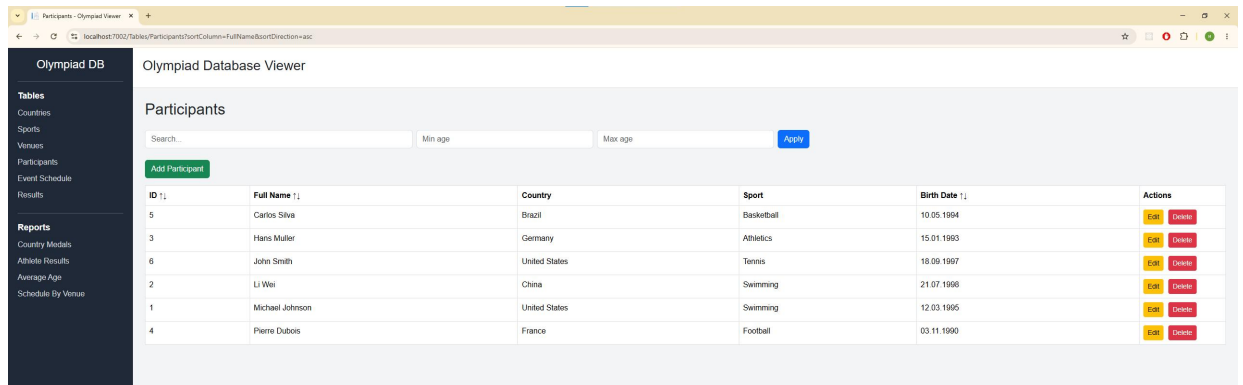
Также реализована фильтрация по числовым и датovým значениям. Например, можно отображать спортсменов только определенного возраста.



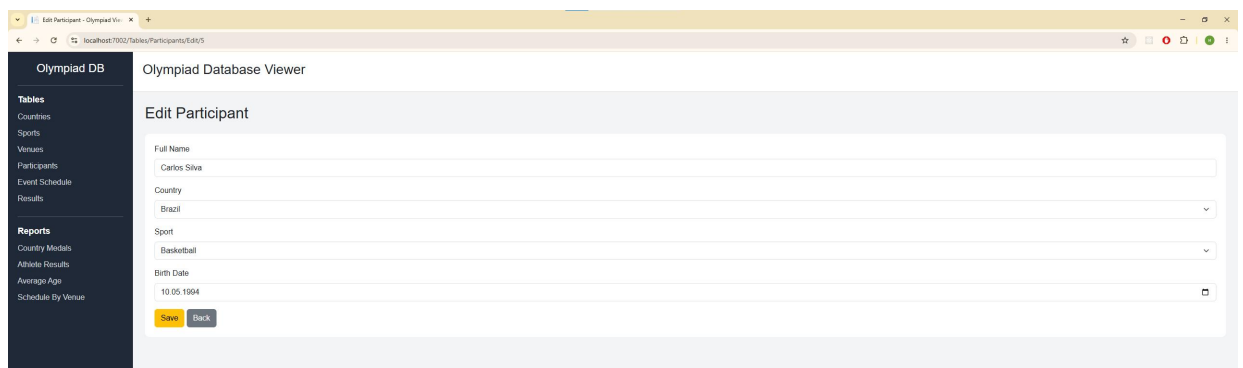
Над каждым столбцом расположены элементы сортировки.

Пользователь может отсортировать данные по возрастанию или убыванию.

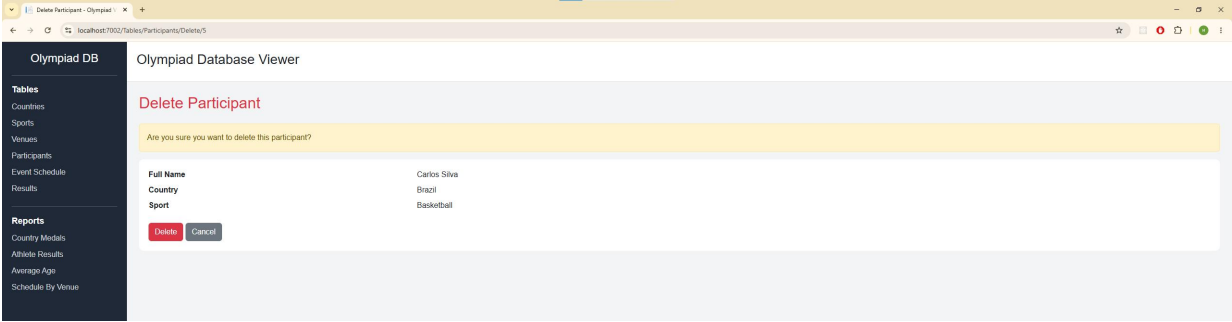
Пример - сортировка по полю Full Name.



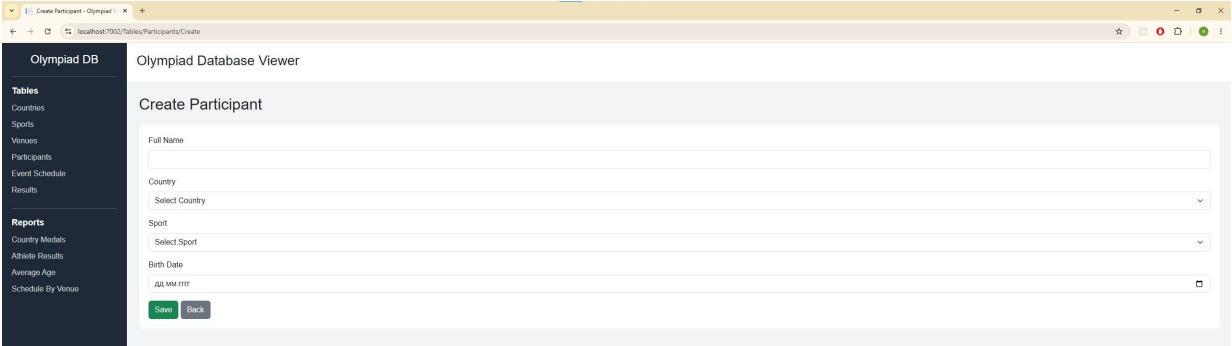
При нажатии кнопки Edit открывается отдельная страница с формой редактирования данных участника.



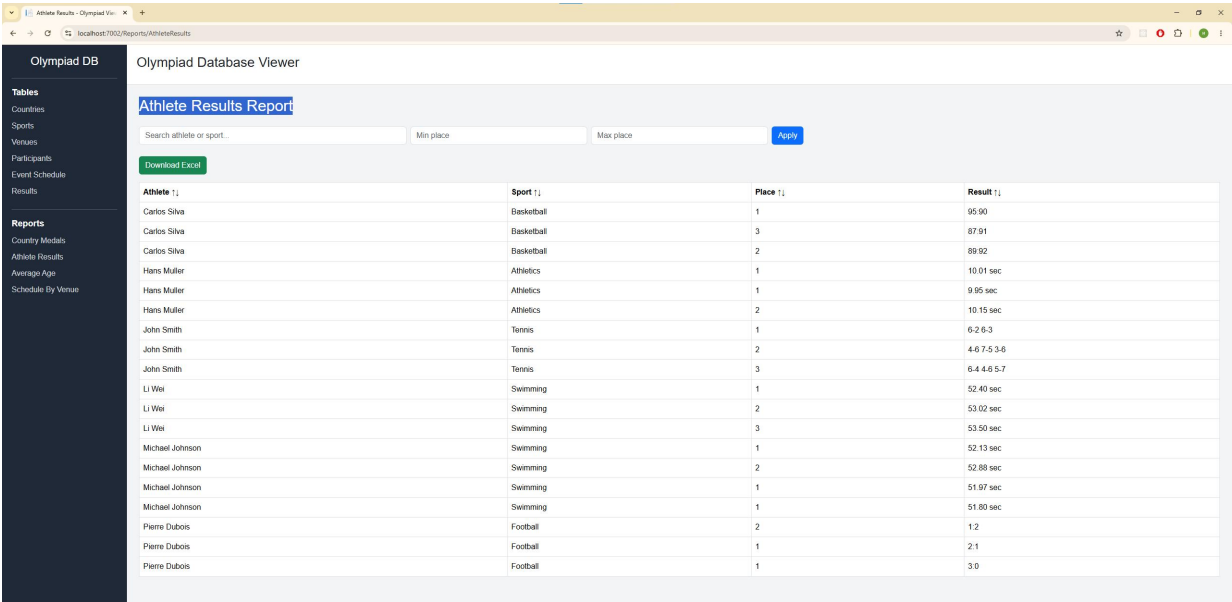
При нажатии кнопки Delete открывается страница подтверждения удаления записи.



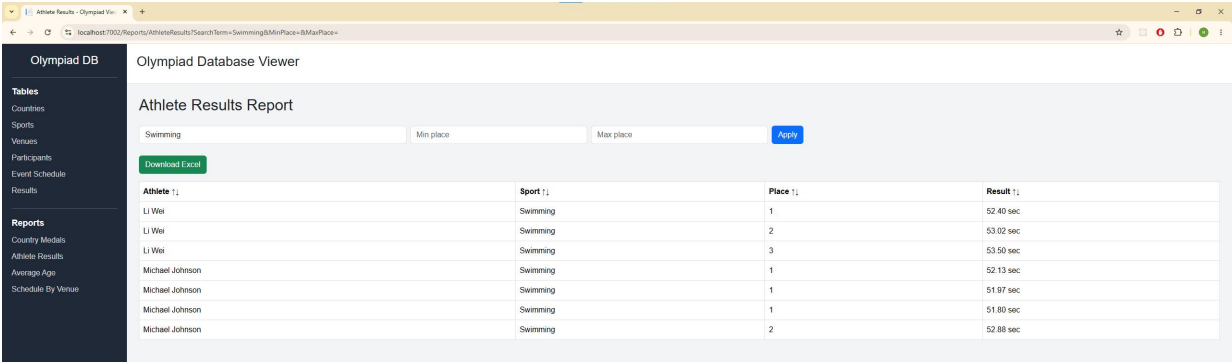
Для добавления новых записей используется кнопка Create New.



Также для примера рассмотрим страницу отчёта Athlete Results Report. Отчёты нельзя редактировать, добавлять или удалять. Данный отчёт отображает результаты спортсменов с указанием вида спорта, занятого места и итогового результата.



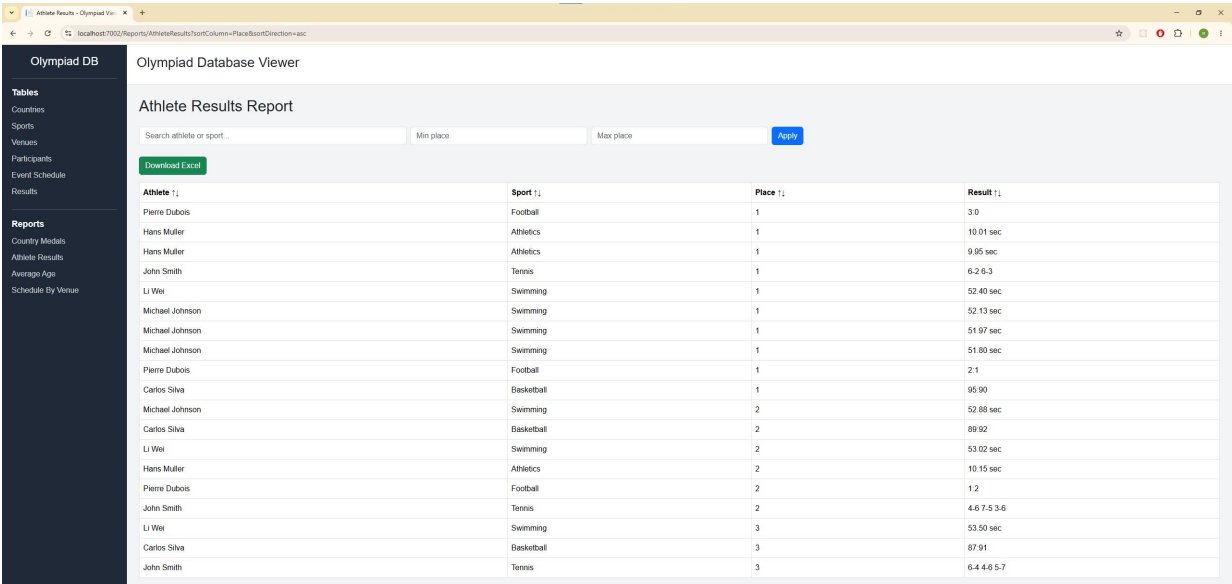
На странице реализован поиск по имени спортсмена и названию вида спорта.



The screenshot shows the 'Olympiad Database Viewer' application. The left sidebar contains a menu with 'Tables' (Countries, Sports, Venues, Participants, Event Schedule, Results) and 'Reports' (Country Medals, Athlete Results, Average Age, Schedule By Venue). The main area is titled 'Athlete Results Report'. It features a search bar with 'Swimming' entered, and filters for 'Min place' and 'Max place'. A 'Download Excel' button is visible. The results table is as follows:

Athlete	Sport	Place	Result
Li Wei	Swimming	1	52.40 sec
Li Wei	Swimming	2	53.02 sec
Li Wei	Swimming	3	53.50 sec
Michael Johnson	Swimming	1	52.13 sec
Michael Johnson	Swimming	1	51.97 sec
Michael Johnson	Swimming	1	51.80 sec
Michael Johnson	Swimming	2	52.88 sec

Реализована фильтрация по по всем основным столбцам, например по занятому месту.



The screenshot shows the 'Olympiad Database Viewer' application with the search bar empty. The results table is as follows:

Athlete	Sport	Place	Result
Pierre Dubois	Football	1	3.0
Hans Muller	Athletics	1	10.01 sec
Hans Muller	Athletics	1	9.95 sec
John Smith	Tennis	1	6.2 6.3
Li Wei	Swimming	1	52.40 sec
Michael Johnson	Swimming	1	52.13 sec
Michael Johnson	Swimming	1	51.97 sec
Michael Johnson	Swimming	1	51.80 sec
Pierre Dubois	Football	1	2.1
Carlos Silva	Basketball	1	95.90
Michael Johnson	Swimming	2	52.88 sec
Carlos Silva	Basketball	2	89.92
Li Wei	Swimming	2	53.02 sec
Hans Muller	Athletics	2	10.15 sec
Pierre Dubois	Football	2	1.2
John Smith	Tennis	2	4.6 7.5 3.6
Li Wei	Swimming	3	53.50 sec
Carlos Silva	Basketball	3	87.91
John Smith	Tennis	3	6.4 4.6 5.7

На странице присутствует кнопка Download Excel, позволяющая экспортировать отчет в файл Excel.

Olympiad DB

Olympiad Database Viewer

Athlete Results Report

Search athlete or sport... Min place Max place Apply

Download Excel

Athlete	Sport	Place	Result
Pierre Dubois	Football	1	3:0
Hans Muller	Athletics	1	10.01 sec
Hans Muller	Athletics	1	9.95 sec
John Smith	Tennis	1	6-2 6-3
Li Wei	Swimming	1	52.40 sec
Michael Johnson	Swimming	1	52.13 sec
Michael Johnson	Swimming	1	51.97 sec
Michael Johnson	Swimming	1	51.80 sec
Pierre Dubois	Football	1	2:1
Carlos Silva	Basketball	1	95:90
Michael Johnson	Swimming	2	52.88 sec
Carlos Silva	Basketball	2	89:92
Li Wei	Swimming	2	53.02 sec
Hans Muller	Athletics	2	10.15 sec
Pierre Dubois	Football	2	1:2
John Smith	Tennis	2	4-6 7-5 3-6
Li Wei	Swimming	3	53.50 sec
Carlos Silva	Basketball	3	87:91
John Smith	Tennis	3	6-4 4-6 5-7

Экспорт выполняется с учётом текущих фильтров и сортировки. Это означает, что пользователь получает именно те данные, которые отображаются на экране.

WPS Office

AthleteResults (1).xlsx

Главная Вставка Разметка страницы Формулы Данные

Обновите подписку, чтобы разблокировать неограниченный функционал для работы с PDF-файлами, расши...

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	ParticipantId	FullName	Country	SportName	StartDate	StartTime	Venue	Place	Result							
2	4	Pierre Dubois	France	Football	15.07.2026	16:00:00	Central Stadium	1	3:0							
3	3	Hans Muller	Germany	Athletics	13.07.2026	09:00:00	Central Stadium	1	10.01 sec							
4	3	Hans Muller	Germany	Athletics	10.07.2026	13:00:00	Central Stadium	1	9.95 sec							
5	6	John Smith	United States	Tennis	14.07.2026	10:00:00	Tennis Center	1	6-2 6-3							
6	2	Li Wei	China	Swimming	12.07.2026	14:00:00	Olympic Swimming Pool	1	52.40 sec							
7	1	Michael Johnson	United States	Swimming	10.07.2026	10:00:00	Olympic Swimming Pool	1	52.13 sec							
8	1	Michael Johnson	United States	Swimming	14.07.2026	13:00:00	Olympic Swimming Pool	1	51.97 sec							
9	1	Michael Johnson	United States	Swimming	17.07.2026	08:00:00	Olympic Swimming Pool	1	51.80 sec							
10	4	Pierre Dubois	France	Football	11.07.2026	16:00:00	Central Stadium	1	2:1							
11	5	Carlos Silva	Brazil	Basketball	13.07.2026	15:00:00	Basketball Arena	1	95:90							
12	1	Michael Johnson	United States	Swimming	12.07.2026	14:00:00	Olympic Swimming Pool	2	52.88 sec							
13	5	Carlos Silva	Brazil	Basketball	11.07.2026	19:00:00	Basketball Arena	2	89:92							
14	2	Li Wei	China	Swimming	10.07.2026	10:00:00	Olympic Swimming Pool	2	53.02 sec							
15	3	Hans Muller	Germany	Athletics	15.07.2026	11:00:00	Central Stadium	2	10.15 sec							
16	4	Pierre Dubois	France	Football	13.07.2026	12:00:00	Central Stadium	2	1:2							
17	6	John Smith	United States	Tennis	16.07.2026	09:00:00	Tennis Center	2	4-6 7-5 3-6							
18	2	Li Wei	China	Swimming	14.07.2026	13:00:00	Olympic Swimming Pool	3	53.50 sec							
19	5	Carlos Silva	Brazil	Basketball	16.07.2026	18:00:00	Basketball Arena	3	87:91							
20	6	John Smith	United States	Tennis	12.07.2026	11:00:00	Tennis Center	3	6-4 4-6 5-7							
21																
22																
23																
24																
25																
26																
27																
28																

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана информационная система для учёта Олимпийских соревнований. Была создана реляционная база данных PostgreSQL с использованием таблиц, внешних ключей, ограничений, индексов, представлений и триггеров. Также было разработано веб-приложение на ASP.NET Core Razor Pages для работы с данными базы. Приложение позволяет просматривать, добавлять, редактировать и удалять записи, выполнять поиск, фильтрацию и сортировку данных. Дополнительно были реализованы аналитические отчёты и экспорт данных в Excel. Разработанная система обеспечивает удобную работу с данными Олимпийских соревнований и демонстрирует практическое применение современных технологий разработки информационных систем.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. PostgreSQL 18.0 Documentation [Электронный ресурс] / The PostgreSQL Global Development Group. – Режим доступа: <https://www.postgresql.org/docs/18/> (дата обращения: 18.05.2026).
2. Руководство по ASP.NET Core 10 [Электронный ресурс] – Режим доступа: <https://metanit.com/sharp/aspnet6/> (дата обращения: 18.05.2026)
3. Руководство по Razor Pages [Электронный ресурс] – Режим доступа: <https://metanit.com/sharp/razorpages/> (дата обращения: 18.05.2026)